

scripts/snapshot-coverage-ledger.sh — User Guide

Last verified: 2026-08-20 (LVA-019 closure, docs/superpowers/plans/2026-08-20-workable-items-backlog-closure.md Task 4)

Inheritance: HelixConstitution §11.4.25 (Full-Automation-Coverage Mandate) + §11.4.18 (script docs)

Overview

Freezes a per-release copy of docs/coverage-ledger.yaml under .lava-ci-evidence/coverage-ledger-snapshots/, mirroring the existing .lava-ci-evidence/distribute-changelog/<channel>/cycle-coverage-map-<version>-<code>.yaml per-release snapshot convention already used by scripts/firebase-distribute.sh for the §6.AK device-coverage gate. This closes the "missing per-release ledgers" half of LVA-019 — the live ledger classifier itself (scripts/check-coverage-ledger.sh --strict) already had no defect; what was missing was a historical record of the ledger's state at each release moment.

What it does

1. Regenerates docs/coverage-ledger.yaml in place via scripts/generate-coverage-ledger.sh --quiet, so the snapshot reflects the current tree, not a stale committed copy.
2. Copies the freshly regenerated ledger to .lava-ci-evidence/coverage-ledger-snapshots/<version>-<code>.yaml.
3. Restores docs/coverage-ledger.yaml to its pre-run committed state (via git checkout) if the regeneration step is what dirtied it, so the script never leaves an unwanted tracked-file diff in the working tree as a side effect of taking a snapshot.

Usage

```
scripts/snapshot-coverage-ledger.sh <version>-<code>
```

Example:

```
scripts/snapshot-coverage-ledger.sh 1.3.17-1085
# -> writes .lava-ci-evidence/coverage-ledger-snapshots/
      1.3.17-1085.yaml
```

Where it's invoked from

scripts/firebase-distribute.sh, immediately after the §6.AK Gate 7 cycle-coverage block (step "1d. LVA-019 — per-release coverage-ledger snapshot"), once per distribute run:

```
bash "$SCRIPT_DIR/snapshot-coverage-ledger.sh" "$APP_VERSION-
      $APP_VERSION_CODE" || \
    echo
      "    WARNING: coverage-ledger snapshot failed (non-fatal,
      distribute continues)"
```

The call is **advisory / non-blocking** — a snapshot failure logs a warning but does not abort the distribute. The coverage ledger's own STRICT gate (scripts/check-coverage-ledger.sh --strict) already runs elsewhere in the pipeline as the actual release gate; this script is a historical-record snapshot, not a gate.

Output

- .lava-ci-evidence/coverage-ledger-snapshots/<version>-<code>.yaml — a byte-for-byte copy of docs/coverage-ledger.yaml at the moment of the run.
- docs/coverage-ledger.yaml — left unchanged in the working tree (regenerated internally to produce fresh snapshot content, then restored if it was clean beforehand).

Companion files

- scripts/generate-coverage-ledger.sh — the generator this script calls to produce fresh ledger content before snapshotting.
- scripts/check-coverage-ledger.sh — the live STRICT/advisory verifier; see docs/scripts/check-coverage-ledger.sh.md.
- docs/coverage-ledger.yaml — the committed canonical ledger this script snapshots.
- scripts/firebase-distribute.sh — the sole caller.

Cross-references

- HelixConstitution Constitution.md §11.4.25 (the mandate)
- docs/superpowers/plans/2026-08-20-workable-items-backlog-closure.md Task 4

- `.lava-ci-evidence/workable-items-closures/2026-08-20-lva-019-closure.md` (LVA-019 closure evidence)
- `Lava CLAUDE.md §6.J/§6.L` (anti-bluff posture: a snapshot mechanism that silently dirties tracked state on every run is itself the kind of side effect the Anti-Bluff Pact exists to catch)

Inheritance + classification

- Inheritance: `HelixConstitution §11.4.25 + §11.4.18 + §6.J/§6.L`
- Classification: project-specific (the Lava distribute-pipeline wiring is project-specific; the `§11.4.25` per-release snapshot discipline is universal per `HelixConstitution`)